

WEEK-2 SKILL

Task-1: To Apply Escape Sequences, Store Command History, Navigate Previous Commands, Navigate Next Commands, Update Input Buffer, Test Recall Functionality

Escape:

```
int main() {
    struct termios oldt, newt;
    char ch;

    tcgetattr(STDIN_FILENO, &oldt);
    newt = oldt;

    newt.c_lflag &= ~(ICANON | ECHO);
    tcsetattr(STDIN_FILENO, TCSANOW, &newt);

    printf("Press an arrow key or 'q' to quit:\n");

    while (1) {
        ch = getchar();

        if (ch == 'q')
            break;

        if (ch == 27) {
            char c1 = getchar();
            char c2 = getchar();

            if (c1 == '[') {
                switch (c2) {
                    case 'A':
                        printf("UP ARROW\n");
                        break;

                    case 'B':
                        printf("DOWN ARROW\n");
                        break;

                    case 'C':
                        printf("RIGHT ARROW\n");
                        break;

                    case 'D':
                        printf("LEFT ARROW\n");
                        break;
                }
            }
        }
    }

    tcsetattr(STDIN_FILENO, TCSANOW, &oldt);

    return 0;
}
```

OUTPUT:

```
pavani@DESKTOP-380LEIN:~/OSSP$ nano week2_task1.c
pavani@DESKTOP-380LEIN:~/OSSP$ nano escape.c
pavani@DESKTOP-380LEIN:~/OSSP$ gcc escape.c -o escape
pavani@DESKTOP-380LEIN:~/OSSP$ ./escape
Press an arrow key or 'q' to quit:
LEFT ARROW
RIGHT ARROW
DOWN ARROW
DOWN ARROW
pavani@DESKTOP-380LEIN:~/OSSP$ █
```

History:

pavani@DESKTOP-38OLEIN: ~/OSSP

GNU nano 8.7.1

history.c *

```
int main() {
    char history[MAX_HISTORY][MAX_LENGTH];
    char command[MAX_LENGTH];
    int count = 0;

    printf("Simple Command History\n");
    printf("Type commands and type 'history' to display them.\n");
    printf("Type 'exit' to quit.\n\n");

    while (1) {
        printf("shell> ");
        fgets(command, MAX_LENGTH, stdin);

        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
            break;

        if (strcmp(command, "history") == 0) {
            printf("\nCommand History:\n");

            for (int i = 0; i < count; i++) {
                printf("%d %s\n", i + 1, history[i]);
            }

            printf("\n");
            continue;
        }

        if (strlen(command) > 0) {
            if (count < MAX_HISTORY) {
                strcpy(history[count], command);
                count++;
            } else {
                /* Remove oldest command */
                for (int i = 1; i < MAX_HISTORY; i++) {
                    strcpy(history[i - 1], history[i]);
                }

                strcpy(history[MAX_HISTORY - 1], command);
            }
        }
    }

    printf("Exiting...\n");

    return 0;
}
```

Output:

```
pavani@DESKTOP-380LEIN:~/OSSP$ nano history.c
pavani@DESKTOP-380LEIN:~/OSSP$ gcc history.c -o history
pavani@DESKTOP-380LEIN:~/OSSP$ ./history
Simple Command History
Type commands and type 'history' to display them.
Type 'exit' to quit.

shell> ls
shell> pwd
shell> date
shell> whoami
shell> history

Command History:
1  ls
2  pwd
3  date
4  whoami

shell>
```

Previous:

pavani@DESKTOP-38OLEIN: ~/OSSP

GNU nano 8.7.1

previous.c *

```
struct termios oldt, newt;
char ch;

tcgetattr(STDIN_FILENO, &oldt);
newt = oldt;

newt.c_lflag &= ~(ICANON | ECHO);
tcsetattr(STDIN_FILENO, TCSANOW, &newt);

printf("Previous Command Navigation\n");
printf("Press UP ARROW to recall previous commands.\n");
printf("Press 'q' to quit.\n\n");

printf("shell> ");
fflush(stdout);

while (1) {
    ch = getchar();

    if (ch == 'q')
        break;

    /* Detect escape sequence */
    if (ch == 27) {
        char c1 = getchar();
        char c2 = getchar();

        if (c1 == '[' && c2 == 'A') {
            /* UP ARROW */

            if (index > 0) {
                index--;

                printf("\r\033[K");
                printf("shell> %s", history[index]);
                fflush(stdout);
            }
        }
    }
}

tcsetattr(STDIN_FILENO, TCSANOW, &oldt);

printf("\nExiting...\n");

return 0;
```

Output:

```
Exiting...
pavani@DESKTOP-380LEIN:~/OSSP$ nano previous.c
pavani@DESKTOP-380LEIN:~/OSSP$ gcc previous.c -o previous
pavani@DESKTOP-380LEIN:~/OSSP$ ./previous
Previous Command Navigation
Press UP ARROW to recall previous commands.
Press 'q' to quit.

shell>
Exiting...
pavani@DESKTOP-380LEIN:~/OSSP$ _
```

Next:

pavani@DESKTOP-38OLEIN: ~/OSSP

GNU nano 8.7.1

next.c *

```
int history_count = 4;
int index = 0;

struct termios oldt, newt;
char ch;

tcgetattr(STDIN_FILENO, &oldt);
newt = oldt;

newt.c_lflag &= ~(ICANON | ECHO);
tcsetattr(STDIN_FILENO, TCSANOW, &newt);

printf("Next Command Navigation\n");
printf("Press DOWN ARROW to navigate next commands.\n");
printf("Press 'q' to quit.\n\n");

printf("Starting command: %s\n", history[index]);

while (1) {
    ch = getchar();

    if (ch == 'q')
        break;

    /* Detect escape sequence */
    if (ch == 27) {
        char c1 = getchar();
        char c2 = getchar();

        if (c1 == '[' && c2 == 'B') {
            /* DOWN ARROW */

            if (index < history_count - 1) {
                index++;

                printf("Next command: %s\n", history[index]);
                fflush(stdout);
            }
        }
    }
}

tcsetattr(STDIN_FILENO, TCSANOW, &oldt);

printf("\nExiting...\n");

return 0;
```


Output:

```
pavani@DESKTOP-380LEIN:~/OSSP$ nano next.c
pavani@DESKTOP-380LEIN:~/OSSP$ gcc next.c -o next
pavani@DESKTOP-380LEIN:~/OSSP$ ./next
Next Command Navigation
Press DOWN ARROW to navigate next commands.
Press 'q' to quit.

Starting command: ls
Next command: pwd
Next command: date
Next command: whoami
```

Input Buffer:

pavani@DESKTOP-38OLEIN: ~

GNU nano 8.7.1

buffer.c *

```
printf("Input Buffer Update\n");
printf("Type something. Use Backspace to edit.\n");
printf("Press Enter to finish.\n\n");
```

```
printf("Input: ");
fflush(stdout);
```

```
while (1) {
    ch = getchar();

    /* Enter */
    if (ch == '\n' || ch == '\r') {
        buffer[position] = '\0';
        break;
    }

    /* Backspace */
    if (ch == 127 || ch == 8) {
        if (position > 0) {
            position--;
            buffer[position] = '\0';

            printf("\b \b");
            fflush(stdout);
        }
        continue;
    }

    /* Normal character */
    if (position < BUFFER_SIZE - 1) {
        buffer[position] = ch;
        position++;

        buffer[position] = '\0';

        putchar(ch);
        fflush(stdout);
    }
}
```

```
tcsetattr(STDIN_FILENO, TCSANOW, &oldt);
```

```
printf("\n\nFinal Input Buffer: %s\n", buffer);
printf("Buffer Length: %d\n", position);
```

```
return 0;
```

```
}_
```

Output:

```
pavani@DESKTOP-380LEIN: ~  
pavani@DESKTOP-380LEIN:~$ nano buffer.c  
pavani@DESKTOP-380LEIN:~$ gcc buffer.c -o buffer  
pavani@DESKTOP-380LEIN:~$ ./buffer  
Input Buffer Update  
Type something. Use Backspace to edit.  
Press Enter to finish.  
  
Input: hello123  
  
Final Input Buffer: hello123  
Buffer Length: 8  
pavani@DESKTOP-380LEIN:~$
```

Recall :

pavani@DESKTOP-38OLEIN: ~

GNU nano 8.7.1

recall.c *

```
#include <stdio.h>
#include <string.h>

#define MAX_HISTORY 10
#define MAX_LENGTH 100

int main() {
    char history[MAX_HISTORY][MAX_LENGTH] = {
        "ls",
        "pwd",
        "date",
        "whoami",
        "gcc program.c"
    };

    int count = 5;
    int choice;

    printf("Command Recall Test\n");
    printf("-----\n");

    printf("\nStored Commands:\n");

    for (int i = 0; i < count; i++) {
        printf("%d. %s\n", i + 1, history[i]);
    }

    printf("\nEnter command number to recall: ");
    scanf("%d", &choice);

    if (choice >= 1 && choice <= count) {
        printf("\nRecalled Command: %s\n", history[choice - 1]);
    } else {
        printf("\nInvalid command number.\n");
    }

    return 0;
}
```

Output:

```
pavani@DESKTOP-380LEIN:~$ nano recall.c
pavani@DESKTOP-380LEIN:~$ gcc recall.c -o recall
pavani@DESKTOP-380LEIN:~$ ./recall
Command Recall Test
-----

Stored Commands:
1. ls
2. pwd
3. date
4. whoami
5. gcc program.c

Enter command number to recall: 3

Recalled Command: date
pavani@DESKTOP-380LEIN:~$
```

Report: In this task, I implemented command history and input buffer operations using C. I used escape sequences to detect special keys like Up Arrow and Down Arrow. The commands entered by the user were stored in the history and could be recalled when needed. I also implemented backspace to update the input buffer. By testing different commands, I was able to check the command storing and recalling functionality successfully.

Task-2: To Allocate Buffers Dynamically, Resize Arrays, Prevent Buffer Overflow, Manage Linked Lists, Release Memory Correctly, Verify with Valgrind

pavani@DESKTOP-38OLEIN: ~/ossp

GNU nano 8.7.1

task2.c *

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define INITIAL_SIZE 5
#define MAX_SIZE 100

/* Linked List Node */
typedef struct Node {
    char command[100];
    struct Node *next;
} Node;

/* Add node to linked list */
void addNode(Node **head, const char *command) {
    Node *newNode = (Node *)malloc(sizeof(Node));

    if (newNode == NULL) {
        printf("Memory allocation failed.\n");
        return;
    }

    strcpy(newNode->command, command);
    newNode->next = NULL;

    if (*head == NULL) {
        *head = newNode;
    } else {
        Node *temp = *head;

        while (temp->next != NULL)
            temp = temp->next;

        temp->next = newNode;
    }
}

/* Display linked list */
void displayList(Node *head) {
    Node *temp = head;

    printf("\nLinked List Commands:\n");

    while (temp != NULL) {
        printf("%s\n", temp->command);
        temp = temp->next;
    }
}
```

Output:

```
pavani@DESKTOP-380LEIN:~/ossp$ nano task2.c
pavani@DESKTOP-380LEIN:~/ossp$ gcc -Wall -g task2.c -o task2
pavani@DESKTOP-380LEIN:~/ossp$ ./task2
=== Dynamic Memory Management ===

1. Dynamic Buffer Allocation
Memory allocated for 5 integers.
Buffer contents: 10 20 30 40 50

2. Resize Array
Array resized to 10 integers.
New buffer contents: 10 20 30 40 50 60 70 80 90 100

3. Buffer Overflow Prevention
Enter an index (0-9): 3
Valid index.
Value at index 3 = 40

4. Linked List Management

Linked List Commands:
ls
pwd
date
whoami

5. Memory Release
All dynamically allocated memory released.

6. Valgrind Verification
Run the following command:
valgrind --leak-check=full ./task2

Program completed successfully.
pavani@DESKTOP-380LEIN:~/ossp$
```

Report: In this task, I worked with dynamic memory allocation in C. I used malloc() to allocate memory and realloc() to increase the size of the array. I also checked the array limits to avoid buffer overflow. A simple linked list was created using dynamic memory. Finally, I used free() to release the allocated memory and checked the program using Valgrind for memory leaks. The program worked successfully.

Session-4:

Task-1: To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output



pavani@DESKTOP-38OLEIN: ~/ossp

GNU nano 8.7.1

task3.c *

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_INPUT 500
#define MAX_TOKENS 100
#define MAX_TOKEN_LENGTH 50

/* Token types */
typedef enum {
    TOKEN_WORD,
    TOKEN_PIPE,
    TOKEN_INPUT,
    TOKEN_OUTPUT,
    TOKEN_APPEND,
    TOKEN_SEMICOLON
} TokenType;

/* Token structure */
typedef struct {
    TokenType type;
    char value[MAX_TOKEN_LENGTH];
} Token;

/* Return token type as string */
const char *getTokenType(TokenType type) {
    switch (type) {
        case TOKEN_WORD:
            return "WORD";

        case TOKEN_PIPE:
            return "PIPE";

        case TOKEN_INPUT:
            return "INPUT_REDIRECT";

        case TOKEN_OUTPUT:
            return "OUTPUT_REDIRECT";

        case TOKEN_APPEND:
            return "APPEND_REDIRECT";

        case TOKEN_SEMICOLON:
            return "SEMICOLON";

        default:
            return "UNKNOWN";
    }
}
```

Output:

```
pavani@DESKTOP-380LEIN:~/ossp$ nano task3.c
pavani@DESKTOP-380LEIN:~/ossp$ gcc -Wall task3.c -o task3
pavani@DESKTOP-380LEIN:~/ossp$ ./task3
=====
      LEXICAL ANALYZER / TOKENIZER
=====

Enter a command:
shell> ls -l test.txt

=== Tokenization Output ===
Token 1:
  Value : ls
  Type  : WORD
Token 2:
  Value : -l
  Type  : WORD
Token 3:
  Value : test.txt
  Type  : WORD

=== Token Validation ===

Token stream is valid.
pavani@DESKTOP-380LEIN:~/ossp$
```

Report: In this task, I implemented tokenization of command input. The input command was divided into different tokens such as words, pipes, and redirection operators. Whitespace was handled while reading the input. I created a token structure to store the token value and its type. The generated tokens were displayed to check whether the input was divided correctly. I also added validation to identify invalid token sequences. The program successfully performed the required tokenization.

Task-4: To Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures



pavani@DESKTOP-38OLEIN: ~/ossp

GNU nano 8.7.1

task4.c *

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_INPUT 500
#define MAX_TOKENS 100
#define MAX_LENGTH 100

/* Token types */
typedef enum {
    WORD,
    PIPE,
    INPUT_REDIRECT,
    OUTPUT_REDIRECT,
    APPEND_REDIRECT
} TokenType;

/* Token structure */
typedef struct {
    TokenType type;
    char value[MAX_LENGTH];
} Token;

/* Command structure */
typedef struct Command {
    char *arguments[MAX_TOKENS];
    int argument_count;

    char *input_file;
    char *output_file;

    int append;
} Command;

/* Display token type */
const char *tokenName(TokenType type) {
    switch (type) {
        case WORD:
            return "WORD";

        case PIPE:
            return "PIPE";

        case INPUT_REDIRECT:
            return "INPUT_REDIRECT";
```

Output:

```
pavani@DESKTOP-380LEIN:~/ossp$ nano task4.c
pavani@DESKTOP-380LEIN:~/ossp$ gcc -Wall -g task4.c -o task4
pavani@DESKTOP-380LEIN:~/ossp$ ./task4

=====
                SIMPLE COMMAND PARSER
=====

Enter command:
shell> ls -l

Tokens:
1. WORD          ls
2. WORD          -l

Syntax is valid.

=====
                PARSE TREE
=====
COMMAND
|
+-- Command 1
   |
   +-- Arguments
       |
       +-- ls
       |
       +-- -l

=====
                EXECUTION STRUCTURE
=====

Command 1:
Arguments: ls -l

Parsing completed successfully.
pavani@DESKTOP-380LEIN:~/ossp$
```

Report: In this task, I implemented a simple parser for processing commands. First, the input was divided into tokens and then the tokens were checked for correct syntax. The program detected errors such as consecutive pipes, missing commands, and missing filenames after redirection symbols. For valid commands, a command structure and parse tree were generated.

Empty commands were also handled. The program successfully parsed the commands and generated the required execution structure.